Lab manual for Database Management Systems (BTS39104)

Department of Computer Science and Engineering
Laboratory Manual
Name of the Programme: B.Tech CSE, Semester III,
Course: Database Management Systems Lab and Code: BTS39104 (P)

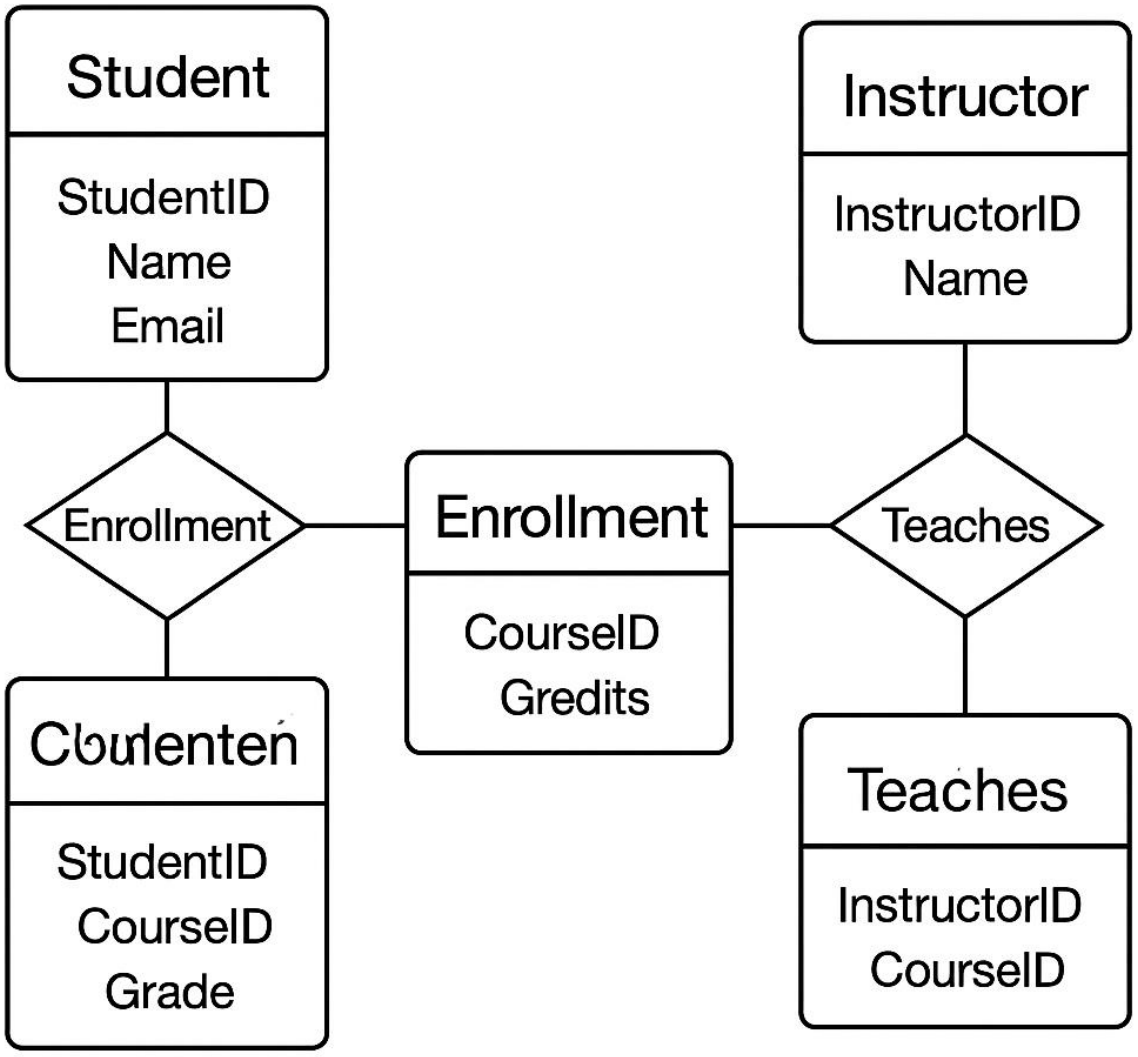
Table of Contents

Experiment/Lab Exercise/Activity No.	Name of the Experiment/Activity/Exercise	Page Number(s)
1	Converting table from ER diagram.	2
2	Creating a college Database, Creating a Student Table, Specifying Relational Data Types, Specifying Constraints, and Creating Indices.	6
3	Employee Management System – Table Creation with Data Types, Constraints, and Indexes	8
4	Employee Management System-Table and Record Handling, INSERT statement	10
5	Employee Management System- SQL Statements – SELECT, INSERT, UPDATE, DELETE, TRUNCATE, DROP, ALTER	12
6	Library Management System- SQL Statements – SELECT, INSERT, UPDATE, DELETE, TRUNCATE, DROP, ALTER	15
7	Employee Management System- Where clause with logical operators	18
8	Employee Management System- Where clause with IN, BETWEEN, LIKE, ORDER BY, GROUP BY and HAVING Clause	20
9	Employee Management System- SQL aggregation function	23
10	Employee Management System- Combining Tables Using JOINS & Subqueries	25
11	Employee Management System- Correlated nested query	28
12	Employee Management System- Views and Operations	31
13	Employee Management System- PL/SQL	33
14	Employee Management System- concept of stored procedure, function, cursor, and trigger.	36
15	Employee Management System- Concept of MongoDB and NoSQL.	39

Module I: Fundamental of DBMS

Assignment I: Converting table from ER diagram.

ER diagram:



Converting an **ER (Entity-Relationship) diagram** to SQL involves translating entities, attributes, and relationships into SQL **CREATE TABLE** statements. Here's a complete guide with examples for:

1. Entities and Attributes
2. Primary Keys
3. One-to-Many Relationships

4. Many-to-Many Relationships
5. Weak Entities
6. Example: Full ER diagram to SQL code

1. Entities and Attributes → SQL Table

ER:

Student (StudentID, Name, Email, DOB)

SQL:

```
CREATE TABLE Student (
   StudentID INT PRIMARY KEY,
   Name VARCHAR(100),
   Email VARCHAR(100) UNIQUE,
   DOB DATE
);
```

2. Primary Key and Composite Key

ER:

Course (CourseID, Title) → Primary Key: CourseID

Enrollment (StudentID, CourseID, Grade) → Composite Key: (StudentID, CourseID)

SQL:

```
CREATE TABLE Course (
   CourseID INT PRIMARY KEY,
   Title VARCHAR(100)
);
CREATE TABLE Enrollment (
   StudentID INT,
   CourseID INT,
   Grade CHAR(1),
   PRIMARY KEY (StudentID, CourseID)
);
```

3. One-to-Many Relationship (Foreign Key)

ER:

Each course is offered by one department.

Department (DeptID, DeptName)

Course (CourseID, Title, DeptID)

SQL:

```
CREATE TABLE Department (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(100)
);
CREATE TABLE Course (
   CourseID INT PRIMARY KEY,
   Title VARCHAR(100),
```

```
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

4. Many-to-Many Relationship

Handled using a **junction table**.

ER:

Students can enroll in multiple courses, and each course can have many students.
Use Enrollment (StudentID, CourseID) as the junction.

SQL (already shown earlier):

```
CREATE TABLE Enrollment (
    StudentID INT,
    CourseID INT,
    Grade CHAR(1),
    PRIMARY KEY (StudentID, CourseID),
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
```

5. Weak Entity

A **weak entity** has no key of its own and depends on another (owner) entity.

ER:

Dependent (EmpID, DependentName, DOB) depends on Employee (EmpID).
Composite key: (EmpID, DependentName)

SQL:

```
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(100)
);
CREATE TABLE Dependent (
    EmpID INT,
    DependentName VARCHAR(100),
    DOB DATE,
    PRIMARY KEY (EmpID, DependentName),
    FOREIGN KEY (EmpID) REFERENCES Employee(EmpID)
);
```

FULL EXAMPLE: ER Diagram to SQL

ER Diagram Entities:

- **Student**(StudentID, Name, Email)
- **Course**(CourseID, Title, Credits)
- **Enrollment**(StudentID, CourseID, Grade) – M:N
- **Instructor**(InstructorID, Name)
- **Teaches**(InstructorID, CourseID) – M:N

SQL Code:

```
CREATE TABLE Student (
```

```
    StudentID INT PRIMARY KEY,
    Name VARCHAR(100),
    Email VARCHAR(100) UNIQUE
);
CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    Title VARCHAR(100),
    Credits INT
);
CREATE TABLE Instructor (
    InstructorID INT PRIMARY KEY,
    Name VARCHAR(100)
);
CREATE TABLE Enrollment (
    StudentID INT,
    CourseID INT,
    Grade CHAR(1),
    PRIMARY KEY (StudentID, CourseID),
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
CREATE TABLE Teaches (
    InstructorID INT,
    CourseID INT,
    PRIMARY KEY (InstructorID, CourseID),
    FOREIGN KEY (InstructorID) REFERENCES Instructor(InstructorID),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
```

Module II: Create Database

Assignment II: Creating a college Database, Creating a Student Table, Specifying Relational Data Types, Specifying Constraints, and Creating Indices.

Topics covered:

- Creating a Database
- Creating a Table
- Specifying Relational Data Types
- Specifying Constraints
- Creating Indices

1. Creating a Database

```
CREATE DATABASE CollegeDB;
USE CollegeDB;
```

2. Creating a Table

Let's create a Students table.

```
CREATE TABLE Students (
   StudentID INT,
   FirstName VARCHAR(50),
   LastName VARCHAR(50),
   DOB DATE,
   Gender CHAR(1),
   DepartmentID INT
);
```

3. Specifying Relational Data Types

Column	Data Type	Description
StudentID	INT	Integer ID for student
FirstName	VARCHAR(50)	First name, up to 50 characters
LastName	VARCHAR(50)	Last name, up to 50 characters
DOB	DATE	Date of Birth
Gender	CHAR(1)	Gender - 'M' or 'F'
DepartmentID	INT	Foreign key reference to Dept.

4. Specifying Constraints

Let's now re-create the table with constraints:

```
CREATE TABLE Students (
   StudentID INT PRIMARY KEY,
   FirstName VARCHAR(50) NOT NULL,
   LastName VARCHAR(50) NOT NULL,
   DOB DATE CHECK (DOB < CURDATE()),
   Gender CHAR(1) CHECK (Gender IN ('M', 'F')),
   DepartmentID INT,
   FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);
```

Constraints Used:

- PRIMARY KEY: Uniquely identifies each record.
- NOT NULL: Field must contain a value.
- CHECK: Validates logical condition.
- FOREIGN KEY: Links to another table.

5. Creating Indices

```
-- Creating a simple index on LastName for faster search
CREATE INDEX idx_lastname ON Students(LastName);

-- Composite index (optional)
CREATE INDEX idx_fullname ON Students(FirstName, LastName);
```

Expected Output

- Database CollegeDB should be created.
- Students table should be structured properly with constraints.
- Indexes should optimize search on selected fields.

Lab Assignment

1. Create a Courses table with appropriate data types and constraints.
2. Add a foreign key relationship to the Students table.
3. Create an index on the course name field.

Module II: Create Database

Assignment III: Employee Management System – Table Creation with Data Types, Constraints, and Indexes

1. Create Database

```
CREATE DATABASE CompanyDB;
USE CompanyDB;
```

2. Create Departments Table

```
CREATE TABLE Departments (
   DepartmentID INT PRIMARY KEY,
   DepartmentName VARCHAR(100) UNIQUE NOT NULL,
   Location VARCHAR(100)
);
```

Explanation:

- DepartmentID: Primary key
- DepartmentName: Unique and cannot be null
- Location: Optional text field

3. Create Employees Table

```
CREATE TABLE Employees (
   EmployeeID INT PRIMARY KEY,
   FirstName VARCHAR(50) NOT NULL,
   LastName VARCHAR(50) NOT NULL,
   Email VARCHAR(100) UNIQUE,
   Phone CHAR(10),
   Salary DECIMAL(10, 2) CHECK (Salary > 0),
   HireDate DATE DEFAULT CURRENT_DATE,
   Gender CHAR(1) CHECK (Gender IN ('M', 'F')),
   DepartmentID INT,
   FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);
```

Explanation of Constraints and Types:

- EmployeeID: Primary key
- Email: Unique constraint
- Phone: Fixed 10-digit character
- Salary: Decimal with 2 precision and must be > 0
- HireDate: Default set to today's date

- Gender: Only 'M' or 'F' allowed
- FOREIGN KEY: Links to Departments

4. Create Indices

```
-- Index on LastName for faster search
CREATE INDEX idx_lastname ON Employees(LastName);

-- Index on Salary for range queries
CREATE INDEX idx_salary ON Employees(Salary);
```

Expected Output:

- Two normalized tables created with constraints.
- Referential integrity established using foreign keys.
- Indexes created to optimize search operations.

Module III: Handling Table

Assignment IV: Employee Management System-Table and Record Handling, INSERT statement

1. Database & Tables Setup

```
CREATE DATABASE CompanyDB;
USE CompanyDB;

CREATE TABLE Departments (
   DepartmentID INT PRIMARY KEY,
   DepartmentName VARCHAR(100) UNIQUE NOT NULL,
   Location VARCHAR(100)
);

CREATE TABLE Employees (
   EmployeeID INT PRIMARY KEY,
   FirstName VARCHAR(50) NOT NULL,
   LastName VARCHAR(50) NOT NULL,
   Email VARCHAR(100) UNIQUE,
   Phone CHAR(10),
   Salary DECIMAL(10, 2) CHECK (Salary > 0),
   HireDate DATE DEFAULT CURRENT_DATE,
   Gender CHAR(1) CHECK (Gender IN ('M', 'F')),
   DepartmentID INT,
   FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);
```

Lab Exercise – INSERT Statements

Step 1: Inserting into Departments Table

```
INSERT INTO Departments (DepartmentID, DepartmentName, Location)
VALUES
(1, 'Human Resources', 'Mumbai'),
(2, 'Finance', 'Delhi'),
(3, 'IT', 'Bangalore'),
(4, 'Marketing', 'Kolkata');
```

Step 2: Inserting into Employees Table

```
INSERT INTO Employees (
   EmployeeID, FirstName, LastName, Email, Phone, Salary, HireDate, Gender, DepartmentID
) VALUES
(101, 'Rahul', 'Sharma', 'rahul.sharma@example.com', '9876543210', 55000.00, '2023-08-01', 'M', 1),
(102, 'Priya', 'Mehta', 'priya.mehta@example.com', '9123456789', 60000.00, DEFAULT, 'F', 3),
(103, 'Amit', 'Kumar', 'amit.kumar@example.com', '9988776655', 70000.00, '2024-01-10', 'M', 2),
(104, 'Sneha', 'Roy', 'sneha.roy@example.com', '9876501234', 80000.00, NULL, 'F', 4);
```

Important Notes:

- Use DEFAULT or NULL for optional/date fields.
- Make sure the DepartmentID in Employees exists in Departments to satisfy the foreign key.
- Violating UNIQUE, NOT NULL, or CHECK constraints will throw an error.

Expected Output:

- Four departments and four employees inserted successfully.
- HireDate for Priya will auto-fill with the current date.
- Sneha's hire date will be NULL (allowed since no NOT NULL constraint).

Module III: Handling Table

Assignment V: Employee Management System- SQL Statements – SELECT, INSERT, UPDATE, DELETE, TRUNCATE, DROP, ALTER

Database Setup Recap

```
CREATE DATABASE CompanyDB;
USE CompanyDB;

CREATE TABLE Departments (
   DepartmentID INT PRIMARY KEY,
   DepartmentName VARCHAR(100) UNIQUE NOT NULL,
   Location VARCHAR(100)
);

CREATE TABLE Employees (
   EmployeeID INT PRIMARY KEY,
   FirstName VARCHAR(50) NOT NULL,
   LastName VARCHAR(50) NOT NULL,
   Email VARCHAR(100) UNIQUE,
   Phone CHAR(10),
   Salary DECIMAL(10, 2) CHECK (Salary > 0),
   HireDate DATE DEFAULT CURRENT_DATE,
   Gender CHAR(1) CHECK (Gender IN ('M', 'F')),
   DepartmentID INT,
   FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);
```

1. INSERT Statement

```
-- Inserting into Departments
INSERT INTO Departments (DepartmentID, DepartmentName, Location)
VALUES (1, 'HR', 'Mumbai');

-- Inserting into Employees
INSERT INTO Employees (
   EmployeeID, FirstName, LastName, Email, Phone, Salary, Gender, DepartmentID
) VALUES (
   101, 'Amit', 'Verma', 'amit.verma@example.com', '9876543210', 50000.00, 'M', 1
);
```

2. SELECT Statement

```
-- Select all records
SELECT * FROM Employees;

-- Select specific columns
SELECT FirstName, Salary FROM Employees;

-- Use WHERE clause
SELECT * FROM Employees WHERE Salary > 40000;
```

3. UPDATE Statement

```
-- Update salary of an employee
UPDATE Employees
SET Salary = 60000
WHERE EmployeeID = 101;

-- Update multiple fields
UPDATE Employees
SET Salary = 65000, Phone = '9999999999'
WHERE Email = 'amit.verma@example.com';
```

4. DELETE Statement

```
-- Delete a specific employee
DELETE FROM Employees
WHERE EmployeeID = 101;

-- Delete all employees in a department
DELETE FROM Employees
WHERE DepartmentID = 1;
```

🛑 Note: Deletes rows one by one and can be rolled back (if inside transaction).

5. TRUNCATE Statement

```
-- Remove all rows from Employees table
TRUNCATE TABLE Employees;
```

🛑 Note: TRUNCATE is faster than DELETE and cannot be rolled back.

6. DROP Statement

```
-- Drop the Employees table
DROP TABLE Employees;
```

```
-- Drop the entire database
DROP DATABASE CompanyDB;
```

🛑 Note: Permanently removes schema and data. Be cautious.

7. ALTER Statement

```
-- Add a new column
ALTER TABLE Departments ADD ManagerName VARCHAR(100);

-- Modify existing column
ALTER TABLE Employees MODIFY Salary DECIMAL(12, 2);

-- Rename column (MySQL 8+)
ALTER TABLE Departments RENAME COLUMN ManagerName TO HeadOfDepartment;

-- Drop a column
ALTER TABLE Departments DROP COLUMN HeadOfDepartment;
```

Expected Output:

- Rows inserted, updated, deleted as instructed.
- Table altered with added/renamed/dropped columns.
- Database/table truncated or dropped successfully.

Viva Questions:

1. Difference between DELETE and TRUNCATE?
2. Can DROP be rolled back?
3. How does ALTER differ from UPDATE?
4. How do constraints affect INSERT and UPDATE?
5. What is the purpose of a WHERE clause?

Lab Assignment:

1. Insert 3 new departments and 5 employees with varying data.
2. Update the salary of employees in a specific department.
3. Delete employees who earn less than 40000.
4. Add a Bonus column to Employees and insert data.
5. Truncate the Departments table and observe the results.

Module III: Handling Table

Assignment VI: Library Management System- SQL Statements – SELECT, INSERT, UPDATE, DELETE, TRUNCATE, DROP, ALTER

Database Setup:

```
CREATE DATABASE LibraryDB;
USE LibraryDB;

CREATE TABLE Books (
   BookID INT PRIMARY KEY,
   Title VARCHAR(100),
   Author VARCHAR(100),
   Genre VARCHAR(50),
   Price DECIMAL(6, 2)
);

CREATE TABLE Members (
   MemberID INT PRIMARY KEY,
   Name VARCHAR(100),
   Email VARCHAR(100) UNIQUE,
   JoinDate DATE DEFAULT CURRENT_DATE
);
```

1. INSERT Statement

```
INSERT INTO Books (BookID, Title, Author, Genre, Price) VALUES
(1, '1984', 'George Orwell', 'Fiction', 299.50),
(2, 'Sapiens', 'Yuval Noah Harari', 'Non-Fiction', 499.99);

INSERT INTO Members (MemberID, Name, Email) VALUES
(101, 'Ravi Kumar', 'ravi.k@example.com'),
(102, 'Anjali Mehra', 'anjali.m@example.com');
```

2. SELECT Statement

```
-- All books
SELECT * FROM Books;

-- Specific fields
SELECT Title, Price FROM Books;
```

```
-- Filter with condition
SELECT * FROM Books WHERE Genre = 'Fiction';
```

3. UPDATE Statement

```
-- Update book price
UPDATE Books
SET Price = 320.00
WHERE BookID = 1;

-- Change member email
UPDATE Members
SET Email = 'ravi.kumar@example.com'
WHERE MemberID = 101;
```

4. DELETE Statement

```
-- Delete a book
DELETE FROM Books WHERE BookID = 2;

-- Delete a member
DELETE FROM Members WHERE MemberID = 102;
```

5. TRUNCATE Statement

```
-- Remove all records from Books
TRUNCATE TABLE Books;
```

6. DROP Statement

```
-- Drop Members table
DROP TABLE Members;
```

7. ALTER Statement

```
-- Add a column to Books
ALTER TABLE Books ADD Publisher VARCHAR(100);

-- Modify Price datatype
ALTER TABLE Books MODIFY Price DECIMAL(8, 2);

-- Rename column (MySQL 8+)
ALTER TABLE Books RENAME COLUMN Genre TO Category;
```

```
-- Drop a column
ALTER TABLE Books DROP COLUMN Publisher;
```

Expected Output:

- Books and members added, modified, deleted successfully.
- Columns altered and tables truncated/dropped as expected.

Module III: Handling Table

Assignment VII: Employee Management System- Where clause with logical operators

Database Setup (Recap)

```
CREATE DATABASE CompanyDB;
USE CompanyDB;

CREATE TABLE Employees (
   EmployeeID INT PRIMARY KEY,
   FirstName VARCHAR(50),
   LastName VARCHAR(50),
   Email VARCHAR(100),
   Phone CHAR(10),
   Salary DECIMAL(10, 2),
   Gender CHAR(1),
   DepartmentID INT
);

INSERT INTO Employees VALUES
(101, 'Amit', 'Verma', 'amit.v@example.com', '9876543210', 55000.00, 'M', 1),
(102, 'Priya', 'Mehra', 'priya.m@example.com', '9123456789', 65000.00, 'F', 2),
(103, 'Ravi', 'Singh', 'ravi.s@example.com', '9988776655', 40000.00, 'M', 1),
(104, 'Sneha', 'Roy', 'sneha.r@example.com', '9876501234', 75000.00, 'F', 3),
(105, 'Anil', 'Kumar', 'anil.k@example.com', '9812345678', 30000.00, 'M', 2);
```

1. Basic SELECT with WHERE Clause

```
-- Select employees with salary more than 50000
SELECT * FROM Employees
WHERE Salary > 50000;
```

Output: Employees earning more than ₹50,000

2. Using AND Logical Operator

```
-- Select male employees in Department 1
SELECT * FROM Employees
WHERE Gender = 'M' AND DepartmentID = 1;
```

Output: Male employees from department 1

3. Using OR Logical Operator

```
-- Select employees in Department 1 or Department 2
SELECT * FROM Employees
WHERE DepartmentID = 1 OR DepartmentID = 2;
```

Output: Employees belonging to department 1 or 2

4. Using NOT Logical Operator

```
-- Select employees who are NOT in Department 3
SELECT * FROM Employees
WHERE NOT DepartmentID = 3;
```

Output: Employees from departments other than 3

5. Combining AND, OR, NOT

```
-- Select female employees who are not in Department 2 and earn more than 60000
SELECT * FROM Employees
WHERE Gender = 'F' AND NOT DepartmentID = 2 AND Salary > 60000;
```

Output: Female employees from other than department 2, with salary > ₹60,000

Module III: Handling Table

Assignment VIII: Employee Management System- Where clause with IN, BETWEEN, LIKE, ORDER BY, GROUP BY and HAVING Clause

Database Setup

```
CREATE DATABASE CompanyDB;
USE CompanyDB;

CREATE TABLE Employees (
   EmployeeID INT PRIMARY KEY,
   FirstName VARCHAR(50),
   LastName VARCHAR(50),
   Email VARCHAR(100),
   Phone CHAR(10),
   Salary DECIMAL(10, 2),
   Gender CHAR(1),
   DepartmentID INT
);

INSERT INTO Employees VALUES
(101, 'Amit', 'Verma', 'amit.v@example.com', '9876543210', 55000.00, 'M', 1),
(102, 'Priya', 'Mehra', 'priya.m@example.com', '9123456789', 65000.00, 'F', 2),
(103, 'Ravi', 'Singh', 'ravi.s@example.com', '9988776655', 40000.00, 'M', 1),
(104, 'Sneha', 'Roy', 'sneha.r@example.com', '9876501234', 75000.00, 'F', 3),
(105, 'Anil', 'Kumar', 'anil.k@example.com', '9812345678', 30000.00, 'M', 2),
(106, 'Kavita', 'Jain', 'kavita.j@example.com', '9123487654', 80000.00, 'F', 3);
```

1. Using IN Clause

```
-- Select employees from Department 1 and 3
SELECT * FROM Employees
WHERE DepartmentID IN (1, 3);
```

Output: Employees in departments 1 or 3

2. Using BETWEEN Clause

```
-- Select employees with salary between ₹40,000 and ₹70,000
SELECT * FROM Employees
WHERE Salary BETWEEN 40000 AND 70000;
```

Output: Employees earning within that salary range (inclusive)

3. Using LIKE Clause

```
-- Select employees whose first name starts with 'A'
SELECT * FROM Employees
WHERE FirstName LIKE 'A%';

-- Select employees whose email contains 'example'
SELECT * FROM Employees
WHERE Email LIKE '%example%';
```

Output: Pattern-matching results using wildcards

4. Using ORDER BY Clause

```
-- Sort employees by salary ascending
SELECT * FROM Employees
ORDER BY Salary ASC;

-- Sort employees by department and then salary (descending)
SELECT * FROM Employees
ORDER BY DepartmentID, Salary DESC;
```

Output: Ordered rows by specified columns

5. Using GROUP BY Clause

```
-- Get total number of employees per department
SELECT DepartmentID, COUNT(*) AS TotalEmployees
FROM Employees
GROUP BY DepartmentID;
```

Output: Summary of employee count per department

6. Using HAVING Clause

```
-- Get departments with more than 1 employee
SELECT DepartmentID, COUNT(*) AS TotalEmployees
FROM Employees
GROUP BY DepartmentID
HAVING COUNT(*) > 1;
```

Output: Filtered group results based on condition

Lab Assignments:

1. Display all female employees with salary between ₹50,000 and ₹80,000.
2. List employees whose names contain "a".
3. Display number of employees in each department sorted by department number.
4. Show departments with an average salary above ₹50,000 using GROUP BY and HAVING.

Module III: Handling Table

Assignment IX: Employee Management System- SQL aggregation function

Database and Table Setup
Create the Tables

```
CREATE DATABASE CompanyDB;
USE CompanyDB;

CREATE TABLE Employees (
   EmployeeID INT PRIMARY KEY,
   FirstName VARCHAR(50),
   LastName VARCHAR(50),
   Salary DECIMAL(10, 2),
   Gender CHAR(1),
   DepartmentID INT
);

CREATE TABLE Departments (
   DepartmentID INT PRIMARY KEY,
   DepartmentName VARCHAR(100)
);
```

Insert Sample Data

```
INSERT INTO Departments VALUES
(1, 'HR'),
(2, 'Finance'),
(3, 'IT');

INSERT INTO Employees VALUES
(101, 'Amit', 'Verma', 50000, 'M', 1),
(102, 'Sneha', 'Roy', 65000, 'F', 3),
(103, 'Ravi', 'Sharma', 45000, 'M', 1),
(104, 'Kavita', 'Jain', 70000, 'F', 3),
(105, 'Anil', 'Mehra', 40000, 'M', 2),
(106, 'Priya', 'Rana', 55000, 'F', 2);
```

SQL Aggregation Functions

1. COUNT() – Total number of employees

SELECT COUNT(*) AS TotalEmployees FROM Employees;

Explanation: Returns the total number of rows in the Employees table.

2. SUM() – Total salary expense

SELECT SUM(Salary) AS TotalSalary FROM Employees;

Explanation: Adds up all the salary values.

3. AVG() – Average employee salary

SELECT AVG(Salary) AS AverageSalary FROM Employees;

Explanation: Computes the average salary of all employees.

4. MAX() – Highest salary

SELECT MAX(Salary) AS HighestSalary FROM Employees;

Explanation: Finds the largest salary value.

5. MIN() – Lowest salary

SELECT MIN(Salary) AS LowestSalary FROM Employees;

Explanation: Finds the smallest salary value.

Group-Based Aggregation
6. Aggregation by Department (GROUP BY)

```
SELECT DepartmentID, COUNT(*) AS EmpCount, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY DepartmentID;
```

Explanation: Shows how many employees are in each department and their average salary.

Module III: Handling Table

Assignment X: Employee Management System- Combining Tables Using JOINS & Subqueries

Database and Table Setup
Create Tables

```
CREATE DATABASE CompanyDB;
USE CompanyDB;

CREATE TABLE Departments (
   DepartmentID INT PRIMARY KEY,
   DepartmentName VARCHAR(100)
);

CREATE TABLE Employees (
   EmployeeID INT PRIMARY KEY,
   FirstName VARCHAR(50),
   LastName VARCHAR(50),
   Salary DECIMAL(10, 2),
   Gender CHAR(1),
   DepartmentID INT,
   FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);
```

Insert Sample Data

```
INSERT INTO Departments VALUES
(1, 'HR'),
(2, 'Finance'),
(3, 'IT');

INSERT INTO Employees VALUES
(101, 'Amit', 'Verma', 50000, 'M', 1),
(102, 'Sneha', 'Roy', 65000, 'F', 3),
(103, 'Ravi', 'Sharma', 45000, 'M', 1),
(104, 'Kavita', 'Jain', 70000, 'F', 3),
(105, 'Anil', 'Mehra', 40000, 'M', 2),
(106, 'Priya', 'Rana', 55000, 'F', 2);
```

Part A: Combining Tables Using JOINS

1. INNER JOIN

```
SELECT E.EmployeeID, E.FirstName, E.LastName, D.DepartmentName
FROM Employees E
INNER JOIN Departments D ON E.DepartmentID = D.DepartmentID;
```

Explanation: Retrieves employees along with their department names (only matching rows).

2. LEFT JOIN

```
SELECT E.FirstName, D.DepartmentName
FROM Employees E
LEFT JOIN Departments D ON E.DepartmentID = D.DepartmentID;
```

Explanation: Shows all employees including those who may not belong to any department.

3. RIGHT JOIN

```
SELECT E.FirstName, D.DepartmentName
FROM Employees E
RIGHT JOIN Departments D ON E.DepartmentID = D.DepartmentID;
```

Explanation: Shows all departments even if no employee belongs to some of them.

4. FULL OUTER JOIN *(Simulated in MySQL using UNION)*

```
SELECT E.FirstName, D.DepartmentName
FROM Employees E
LEFT JOIN Departments D ON E.DepartmentID = D.DepartmentID

UNION

SELECT E.FirstName, D.DepartmentName
FROM Employees E
RIGHT JOIN Departments D ON E.DepartmentID = D.DepartmentID;
```

Explanation: Combines LEFT and RIGHT JOIN results to include all employees and all departments.

Part B: Subqueries

1. Single-Row Subquery

```
-- Employees who earn more than the average salary
SELECT * FROM Employees
WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```

Explanation: Retrieves employees whose salary is above the average.

2. Multi-Row Subquery using IN

```
-- Employees in departments that have more than one employee
SELECT * FROM Employees
WHERE DepartmentID IN (
   SELECT DepartmentID
   FROM Employees
   GROUP BY DepartmentID
   HAVING COUNT(*) > 1
);
```

Explanation: Finds employees in departments with more than one staff.

3. Correlated Subquery

```
-- Employees who earn the maximum salary in their department
SELECT * FROM Employees E1
WHERE Salary = (
   SELECT MAX(Salary)
   FROM Employees E2
   WHERE E1.DepartmentID = E2.DepartmentID
);
```

Explanation: Retrieves employees earning the highest in each department.

Module III: Handling Table

Assignment XI: Employee Management System- Correlated nested query

Assume we have the following tables:

```
CREATE TABLE Department (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50)
);

CREATE TABLE Employee (
   EmpID INT PRIMARY KEY,
   EmpName VARCHAR(50),
   Salary DECIMAL(10, 2),
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

Department Table - Sample Data

```
INSERT INTO Department (DeptID, DeptName) VALUES
(1, 'HR'),
(2, 'Finance'),
(3, 'IT'),
(4, 'Marketing');
```

Employee Table - Sample Data

```
INSERT INTO Employee (EmpID, EmpName, Salary, DeptID) VALUES
(101, 'Alice', 30000, 1),
(102, 'Bob', 25000, 1),
(103, 'Charlie', 50000, 2),
(104, 'David', 48000, 2),
(105, 'Eva', 60000, 2),
(106, 'Frank', 55000, 3),
(107, 'Grace', 70000, 3),
(108, 'Helen', 50000, 3),
(109, 'Ivy', 45000, 4),
(110, 'Jack', 47000, 4);
```

1 Find employees who earn more than the average salary of their department.

```
SELECT EmpName, Salary, DeptID
FROM Employee E1
WHERE Salary > (
   SELECT AVG(Salary)
   FROM Employee E2
   WHERE E1.DeptID = E2.DeptID
);
```

2 List employees who are the highest paid in their respective departments.

```
SELECT EmpName, Salary, DeptID
FROM Employee E1
WHERE Salary = (
   SELECT MAX(Salary)
   FROM Employee E2
   WHERE E1.DeptID = E2.DeptID
);
```

3 Display departments with more than 2 employees.

```
SELECT DeptName
FROM Department D
WHERE (
   SELECT COUNT(*)
   FROM Employee E
   WHERE E.DeptID = D.DeptID
) > 2;
```

4 List employees whose salary is above the overall average but not the highest in their department.

```
SELECT EmpName, Salary
FROM Employee E1
WHERE Salary > (
   SELECT AVG(Salary) FROM Employee
)
AND Salary < (
   SELECT MAX(Salary)
   FROM Employee E2
   WHERE E1.DeptID = E2.DeptID
```

```
);
```

5 Find employees who do not belong to departments having any employee with salary less than 20000.

```
SELECT EmpName
FROM Employee E1
WHERE NOT EXISTS (
   SELECT 1
   FROM Employee E2
   WHERE E1.DeptID = E2.DeptID AND E2.Salary < 20000
);
```

Module IV: Create Virtual Table

Assignment XII: Employee Management System- Views and Operations

1 Creating a View
Let's create a **view** that shows the employee's name, department name, and salary:

```
CREATE VIEW Employee_Details AS
SELECT E.EmpID, E.EmpName, D.DeptName, E.Salary
FROM Employee E
JOIN Department D ON E.DeptID = D.DeptID;
```

This view joins the Employee and Department tables to display consolidated information.

2 Querying the View
You can query the view like a table:

```
SELECT * FROM Employee_Details;
```

Example: List employees earning more than ₹50,000:

```
SELECT EmpName, DeptName, Salary
FROM Employee_Details
WHERE Salary > 50000;
```

3 Updating Data via View
You can update base table data **through the view** (as long as the view allows updates):

Example: Give a ₹2000 raise to all employees in the 'IT' department:

```
UPDATE Employee_Details
SET Salary = Salary + 2000
WHERE DeptName = 'IT';
```

This will update the Employee table behind the view.

4 Inserting Data via View (if possible)
If the view includes **all NOT NULL columns** from the base table, insert is allowed:

```
INSERT INTO Employee_Details (EmpID, EmpName, DeptName, Salary)
VALUES (111, 'Kevin', 'HR', 32000);
```

However, since DeptName is from a joined table, direct insert **won't work** here → must insert into Employee and Department separately.

5 Dropping a View
To delete the view:

```
DROP VIEW Employee_Details;
```

Bonus View Example: High Earners
A view to list employees earning above the average salary of their department:

```
CREATE VIEW High_Earners AS
SELECT E.EmpID, E.EmpName, E.Salary, D.DeptName
FROM Employee E
JOIN Department D ON E.DeptID = D.DeptID
WHERE E.Salary > (
   SELECT AVG(Salary)
   FROM Employee E2
   WHERE E2.DeptID = E.DeptID
);
```

You can then query:

```
SELECT * FROM High_Earners;
```

Module V: Understanding PL/SQL

Assignment XIII: Employee Management System- PL/SQL

Database Schema
We will use these tables:

```
CREATE TABLE Department (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50)
);

CREATE TABLE Employee (
   EmpID INT PRIMARY KEY,
   EmpName VARCHAR(50),
   Salary DECIMAL(10,2),
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

Insert sample data (same as I provided earlier).

PL/SQL Lab Exercises

1 Basic PL/SQL Block: Display Employee Name

```
DECLARE
   v_EmpName Employee.EmpName%TYPE;
BEGIN
   SELECT EmpName INTO v_EmpName
   FROM Employee
   WHERE EmpID = 101;
   DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_EmpName);
END;
/
```

2 IF-ELSE Control Structure: Check Salary

```
DECLARE
   v_Salary Employee.Salary%TYPE;
BEGIN
   SELECT Salary INTO v_Salary
   FROM Employee
```

```
  WHERE EmpID = 101;
  IF v_Salary >= 50000 THEN
    DBMS_OUTPUT.PUT_LINE('High Earner');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Normal Earner');
  END IF;
END;
/
```

3 LOOP Structure: Display All Employee Names

```
DECLARE
  CURSOR emp_cursor IS SELECT EmpName FROM Employee;
  v_name Employee.EmpName%TYPE;
BEGIN
  OPEN emp_cursor;
  LOOP
    FETCH emp_cursor INTO v_name;
    EXIT WHEN emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Employee: ' || v_name);
  END LOOP;
  CLOSE emp_cursor;
END;
/
```

4 EXCEPTION Handling: Handle No Data Found

```
DECLARE
  v_name Employee.EmpName%TYPE;
BEGIN
  SELECT EmpName INTO v_name
  FROM Employee
  WHERE EmpID = 999; -- Non-existent ID
  DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_name);
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No Employee found with given ID');
END;
/
```

5 Procedure: Increase Salary

```
CREATE OR REPLACE PROCEDURE Increase_Salary(p_EmpID INT, p_Amount DECIMAL) AS
BEGIN
```

```
    UPDATE Employee
    SET Salary = Salary + p_Amount
    WHERE EmpID = p_EmpID;
    DBMS_OUTPUT.PUT_LINE('Salary updated for Employee ID: ' || p_EmpID);
END Increase_Salary;
/
```

Call the procedure:

```
BEGIN
    Increase_Salary(101, 2000);
END;
/
```

6 Function: Get Department Name by Employee ID

```
CREATE OR REPLACE FUNCTION Get_DeptName(p_EmpID INT) RETURN VARCHAR IS
    v_DeptName Department.DeptName%TYPE;
BEGIN
    SELECT D.DeptName INTO v_DeptName
    FROM Employee E JOIN Department D ON E.DeptID = D.DeptID
    WHERE E.EmpID = p_EmpID;
    RETURN v_DeptName;
END Get_DeptName;
/
```

Use the function:

```
DECLARE
    v_dept VARCHAR(50);
BEGIN
    v_dept := Get_DeptName(101);
    DBMS_OUTPUT.PUT_LINE('Department: ' || v_dept);
END;
/
```

Module V: Understanding PL/SQL

Assignment XIV: Employee Management System- concept of stored procedure, function, cursor, and trigger.

1 STORED PROCEDURE

Aim: Create a procedure to increase an employee's salary by a given amount.

```
CREATE OR REPLACE PROCEDURE Increase_Salary (
   p_EmpID IN Employee.EmpID%TYPE,
   p_Amount IN Employee.Salary%TYPE
) AS
BEGIN
   UPDATE Employee
   SET Salary = Salary + p_Amount
   WHERE EmpID = p_EmpID;
   DBMS_OUTPUT.PUT_LINE('Salary updated for Employee ID: ' || p_EmpID);
END;
/
```

Execution:

```
BEGIN
   Increase_Salary(101, 2000);
END;
/
```

2 FUNCTION

Aim: Create a function to return an employee's department name.

```
CREATE OR REPLACE FUNCTION Get_DeptName (
   p_EmpID IN Employee.EmpID%TYPE
) RETURN VARCHAR IS
   v_DeptName Department.DeptName%TYPE;
BEGIN
   SELECT D.DeptName INTO v_DeptName
   FROM Employee E JOIN Department D
   ON E.DeptID = D.DeptID
   WHERE E.EmpID = p_EmpID;
   RETURN v_DeptName;
```

```
END;
/
```

Calling function:

```
DECLARE
   v_Dept VARCHAR(50);
BEGIN
   v_Dept := Get_DeptName(101);
   DBMS_OUTPUT.PUT_LINE('Department: ' || v_Dept);
END;
/
```

3 CURSOR

Aim: Use a cursor to display all employee names and salaries.

```
DECLARE
   CURSOR emp_cursor IS
      SELECT EmpName, Salary FROM Employee;
   v_name Employee.EmpName%TYPE;
   v_salary Employee.Salary%TYPE;
BEGIN
   OPEN emp_cursor;
   LOOP
      FETCH emp_cursor INTO v_name, v_salary;
      EXIT WHEN emp_cursor%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE('Employee: ' || v_name || ', Salary: ' || v_salary);
   END LOOP;
   CLOSE emp_cursor;
END;
/
```

4 TRIGGER

Aim: Create a trigger to automatically log salary updates.
Create a log table:

```
CREATE TABLE Salary_Log (
   LogID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   EmpID INT,
   OldSalary DECIMAL(10,2),
```

```
    NewSalary DECIMAL(10,2),
    UpdatedOn DATE
);
```

Create trigger:

```
CREATE OR REPLACE TRIGGER trg_salary_update
AFTER UPDATE OF Salary ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Salary_Log (EmpID, OldSalary, NewSalary, UpdatedOn)
    VALUES (:OLD.EmpID, :OLD.Salary, :NEW.Salary, SYSDATE);
END;
/
```

Now, every salary update will automatically insert a record into Salary_Log.

HOW TO TEST THE TRIGGER:

```
UPDATE Employee
SET Salary = Salary + 1000
WHERE EmpID = 101;

SELECT * FROM Salary_Log;
```

Module V: Understanding PL/SQL

Assignment XV: Employee Management System- Concept of MongoDB and NoSQL.

1 Introduction to NoSQL:

NoSQL = "Not Only SQL" → a class of **non-relational databases** designed for:

- Flexibility with **semi-structured/unstructured data**
- **Scalability** (horizontal scaling)
- No rigid schema (schema-less or schema-on-read)

Common NoSQL types:

Type	Example	Use Case
Document Store	**MongoDB**	Employee data (JSON-like documents)
Key-Value	Redis	Session caching
Column Family	Cassandra	Big data logs
Graph DB	Neo4j	Employee hierarchy relationships

2 Why MongoDB for Employee Management System?

MongoDB is a **document-oriented NoSQL database**, stores data as **BSON (Binary JSON)**.

In Employee Management:

- Store **employee records as documents**
- Flexible schema → no need to predefine columns
- Nested structures (address, contacts) directly embedded

3 Example Employee Document:

```
{
  "_id": 101,
  "EmpName": "Alice",
  "Salary": 30000,
  "Dept": {
    "DeptID": 1,
    "DeptName": "HR"
  },
  "Skills": ["Communication", "Recruitment"],
  "JoinDate": "2023-01-15"
}
```

Each employee is a **document** in the **"employees" collection**.

4 MongoDB Operations for Employee Management:

➤ **Insert:**

```
db.employees.insertOne({
  _id: 101,
  EmpName: "Alice",
  Salary: 30000,
  Dept: { DeptID: 1, DeptName: "HR" },
  Skills: ["Communication", "Recruitment"],
  JoinDate: ISODate("2023-01-15")
});
```

➤ **Query: Find all employees in HR department**

```
db.employees.find({ "Dept.DeptName": "HR" });
```

➤ **Query: Find employees earning > 40000**

```
db.employees.find({ Salary: { $gt: 40000 } });
```

➤ **Update: Increase salary of EmpID 101 by 2000**

```
db.employees.updateOne(
  { _id: 101 },
  { $inc: { Salary: 2000 } }
);
```

➤ **Delete: Remove an employee**

```
db.employees.deleteOne({ _id: 101 });
```

5 Differences: SQL vs NoSQL (MongoDB)

SQL (Relational)	NoSQL (MongoDB)
Tables (Rows/Cols)	Collections (Documents)
Fixed schema	Dynamic schema
JOINs across tables	Embedded/Nested docs
ACID transactions	BASE (eventual consistency)

6 Use Case Mapping:

SQL Entity	MongoDB Equivalent
Employee table	employees collection

SQL Entity	MongoDB Equivalent
Department table	embedded document inside employee

No foreign keys or JOINs → denormalization by embedding dept info inside employee document.

7 Advantages of MongoDB for Employee System:

- ✅ Easy to store **complex/nested data** without JOINs
- ✅ Schema changes **without migration**
- ✅ Scales easily for large organizations
- ✅ Faster reads for aggregated embedded data

8 Sample Lab Exercise Tasks:

- ✅ Insert 5 employee documents
- ✅ Query all employees from "Finance"
- ✅ Update salary for a specific employee
- ✅ Delete an employee
- ✅ Add a new skill to an employee's skill array